

Lab 11 - Task 1 - Single Responsibility Principle (SRP)

The Single Responsibility Principle (SRP) is a fundamental principle of object-oriented design that states that a class should have single responsibility that it does well.

Example 1 of SRP Violation

```
class Report
{
    String title;
    String body;

    // renders report to HTML string
    void generateHTML() { }

    // connects to DB and persists report
    void saveToDatabase() { }
}
```

Explanation of Violation:

`Report` handles both HTML generation and database access. Changing how reports are displayed or how they are stored would both force modifications to the same class.

Refactored Code Following SRP

```
class Report
{
    String title;
    String body;
    // getters and data-only logic
}

class HtmlReportRenderer
{
    // returns HTML string representation of the report
    String render(Report report) { }
}

class ReportDatabaseRepository
{
    // connects to DB and saves the report
    void save(Report report) { }
}
```

Explanation of Fix:

Now, `Report` is only responsible for holding data. `HtmlReportRenderer` is responsible for rendering the report to HTML, and `ReportDatabaseRepository` is responsible for saving the report to the database. Each class has a single responsibility,

Example 2 of SRP Violation

```
class Employee
{
    // salary calculations
}
```

```
void calculatePay() { }

// formats and prints payslip
void printPayslip() { }
}
```

Explanation of Violation:

`Employee` has two responsibilities; payment calculation and payslip generation. A change in tax calculation or a change in the payslip layout would both affect the same class, violating SRP.

Refactored Code Following SRP

```
class Employee
{
    Money baseSalary;
    int overtimeHours;
    // data and minimal behavior
}

class PayCalculator
{
    // calculates pay based on employee data
    Money calculatePay(Employee e) { }
}

class PayslipGenerator
{
    // generates payslip based on employee data and calculated pay
    void printPayslip(Employee e, Money pay) { }
}
```

Explanation of Fix:

`Employee` is only responsible for holding employee data. `PayCalculator` is only responsible for calculating pay, and `PayslipGenerator` is responsible for generating the payslip. Each class has a single responsibility, adhering to SRP.

Lab 11 - Task 1 - Open/Closed Principle (OCP)

The Open/Closed Principle (OCP) states that software entities (classes, modules, functions, etc.) should be open for extension but closed for modification. This means that you should be able to add new functionality without changing existing code, which helps maintaining stability.

Example 1 of OCP Violation

```
class Circle
{
    double radius;
}

class Rectangle
{
    double width;
    double height;
}

class AreaCalculator
{
    double calculateArea(Object shape)
    {
        if (shape instanceof Circle c)
        {
            return Math.PI * c.radius * c.radius;
        }
        else if (shape instanceof Rectangle r)
        {
            return r.width * r.height;
        }
        return 0;
    }
}
```

Explanation of Violation:

`AreaCalculator` uses type checking to determine the shape type and calculate the area. If we want to add a new shape, we would need to modify the `calculateArea()` method in `AreaCalculator`, which violates OCP.

Refactored Code Following OCP

```
interface Shape
{
    double area();
}

class Circle implements Shape
{
    double radius;

    @Override
    public double area() { return Math.PI * radius * radius; }
}
```

```

class Rectangle implements Shape
{
    double width;
    double height;

    @Override
    public double area() { return width * height; }
}

class AreaCalculator
{
    double calculateArea(Shape shape)
    {
        return shape.area(); // no down-casting or if conditions to determine type
    }
}

```

Explanation of Fix:

The `AreaCalculator` is closed for modification because it works with the `Shape` interface. Adding a new shape means implementing the `Shape` interface, an extension without changing existing code.

Example 2 of OCP Violation

```

class Logger
{
    void log(String message, String target)
    {
        if (target.equals("console"))
        {
            System.out.println(message);
        }
        else if (target.equals("file"))
        {
            // write to file
        }
    }
}

```

Explanation of Violation:

`Logger` uses conditional logic to determine where to log the message. Adding a new logging target would require modifying the `log()` method, which violates OCP.

Refactored Code Following OCP

```

import java.util.List;

interface LogDestination
{
    void write(String message);
}

```

```
class ConsoleDestination implements LogDestination
{
    public void write(String message) { System.out.println(message); }
}

class FileDestination implements LogDestination
{
    // writes to the file
    public void write(String message) { }
}

class Logger
{
    private List<LogDestination> destinations;

    void log(String message)
    {
        for (LogDestination dest : destinations)
            dest.write(message);
    }
}
```

Explanation of Fix:

`Logger` is now closed for modification because it works with the `LogDestination` interface. Adding a new logging target means implementing the `LogDestination` interface, an extension without changing existing code.

Lab 11 - Task 1 - Liskov Substitution Principle (LSP)

The Liskov Substitution Principle (LSP) states that objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program. In other words, subclasses should be able to substitute their base class without causing errors or unexpected behavior.

Example 1 of LSP Violation

```
class Bird
{
    void fly()
    {
        System.out.println("Flying");
    }
}

class Ostrich extends Bird
{
    @Override
    void fly()
    {
        throw new UnsupportedOperationException("Ostrich can't fly");
    }
}
```

Explanation of Violation:

In this example, `Ostrich` is a subclass of `Bird`, but it cannot fly. If we substitute an `Ostrich` for a `Bird`, it would cause an error when calling the `fly()` method, violating LSP.

Refactored Code Following LSP

```
abstract class Bird
{
    abstract void move();
}

class FlyingBird extends Bird
{
    @Override
    void move() { fly(); }

    // flying logic
    void fly() { }
}

class Ostrich extends Bird
{
    @Override
    void move() { run(); }

    // running logic
    void run() { }
}
```

Explanation of Fix:

Now, `Bird` has an abstract method `move()`, and both `FlyingBird` and `Ostrich` implement it according to their capabilities. Substituting an `Ostrich` for a `Bird` will not cause errors, as both classes adhere to the contract defined by the `Bird` class, thus following LSP.

Example 2 of LSP Violation

```
class Rectangle
{
    int width, height;

    void setWidth(int w) { width = w; }

    void setHeight(int h) { height = h; }

    int getArea() { return width * height; }
}

class Square extends Rectangle
{
    @Override
    void setWidth(int w)
    {
        super.setWidth(w);
        super.setHeight(w); // forces square constraint
    }

    @Override
    void setHeight(int h)
    {
        super.setHeight(h);
        super.setWidth(h);
    }
}
```

Explanation of Violation:

In this example, `Square` is a subclass of `Rectangle`, but it violates the expected behavior of a rectangle. If we substitute a `Square` for a `Rectangle`, it would cause unexpected behavior when setting width and height, violating LSP.

Refactored Code Following LSP

```
interface Shape
{
    int getArea();
}

class Rectangle implements Shape
{
    int width, height;

    void setWidth(int w) { width = w; }
```

```

    void setHeight(int h) { height = h; }

    @Override
    public int getArea() { return width * height; }
}

class Square implements Shape
{
    int side;

    void setSide(int s) { side = s; }

    @Override
    public int getArea() { return side * side; }
}

```

Explanation of Fix:

Now, both `Rectangle` and `Square` implement the `Shape` interface, which defines the contract for calculating area. Each class implements the `getArea()` method according to its own properties, and substituting one for the other will not cause errors or unexpected behavior, thus adhering to LSP.